

Third Year Engineering

18BTCS601-Design and Analysis of Algorithms

Class - T.Y. (SEM-I)

Unit - V

NP Hard and NP complete problems



Introduction

- We saw in previous units, There are problems that are difficult to solve algorithmically.
- At the same time, some of them are so important that we cannot just sigh in resignation and do nothing.
- This unit outlines several ways of dealing with such difficult problems.

Introduction

- The other distinction between backtracking and branch-and-bound lies in the order in which nodes of the state-space tree are generated.
- For backtracking, this tree is usually developed depth-first (i.e., similar to DFS).
- Branch-and-bound can generate nodes according to several rules: the most natural one is the so-called best-first rule explained

Backtracking

- The principal idea is to construct solutions one component at a time and evaluate such partially constructed candidates as follows.
- If a partially constructed solution can be developed further without violating the problem's constraints, it is done by taking the first remaining legitimate option for the next component.
- If there is no legitimate option for the next component, no alternatives for *any* remaining component need to be considered.
- In this case, the algorithm backtracks to replace the last component of the partially constructed solution with its next option.

Backtracking

- It is convenient to implement this kind of processing by constructing a tree of choices being made, called the ***state-space tree***.
- Its root represents an initial state before the search for a solution begins.
- The nodes of the first level in the tree represent the choices made for the first component of a solution, the nodes of the second level represent the choices for the second component, and so on.
- A node in a state-space tree is said to be ***promising*** if it corresponds to a partially constructed solution that may still lead to a complete solution; otherwise it is called ***nonpromising***

***n*-Queens Problem**

- In backtracking, for n queen problem it should be pointed out that a single solution to the n -queens problem for any $n \geq 4$ can be found in linear time.

NP Class Problems

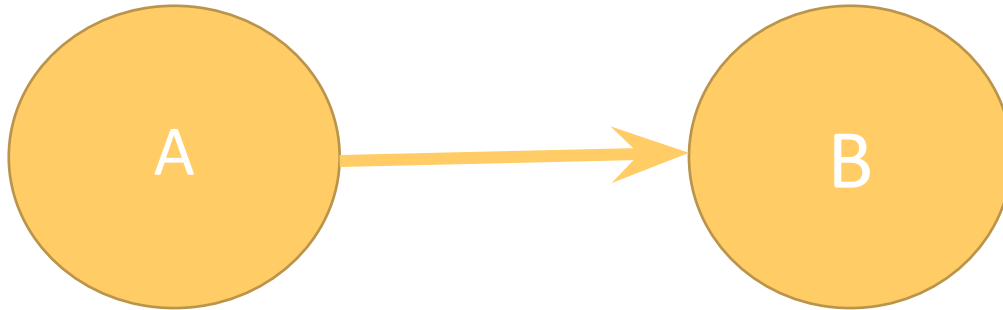
- NP class problems are hard to implement
- Easy to verify
- Exponential in nature
- Are verified in polynomial time
- Non-tractable problems
- Security problems are of type NP

P-Class Problems

- P class problems are easy to implement
- Easy to verify
- Polynomial in nature
- Tractable problems

•NP
•P

Reduction

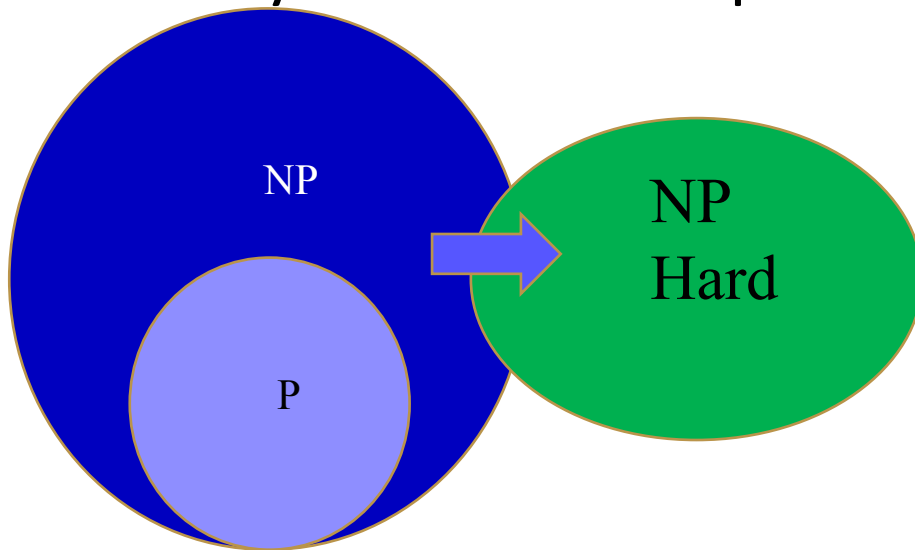


Let A and B are two problems then problem A reduces to problem B iff there is a way to solve A by deterministic algorithm that solves B in polynomial time.

- If A is reducible to B then $A \propto B$.
- If A is reducible to B and B in P then A in P.
- If A not in P implies B not in P.

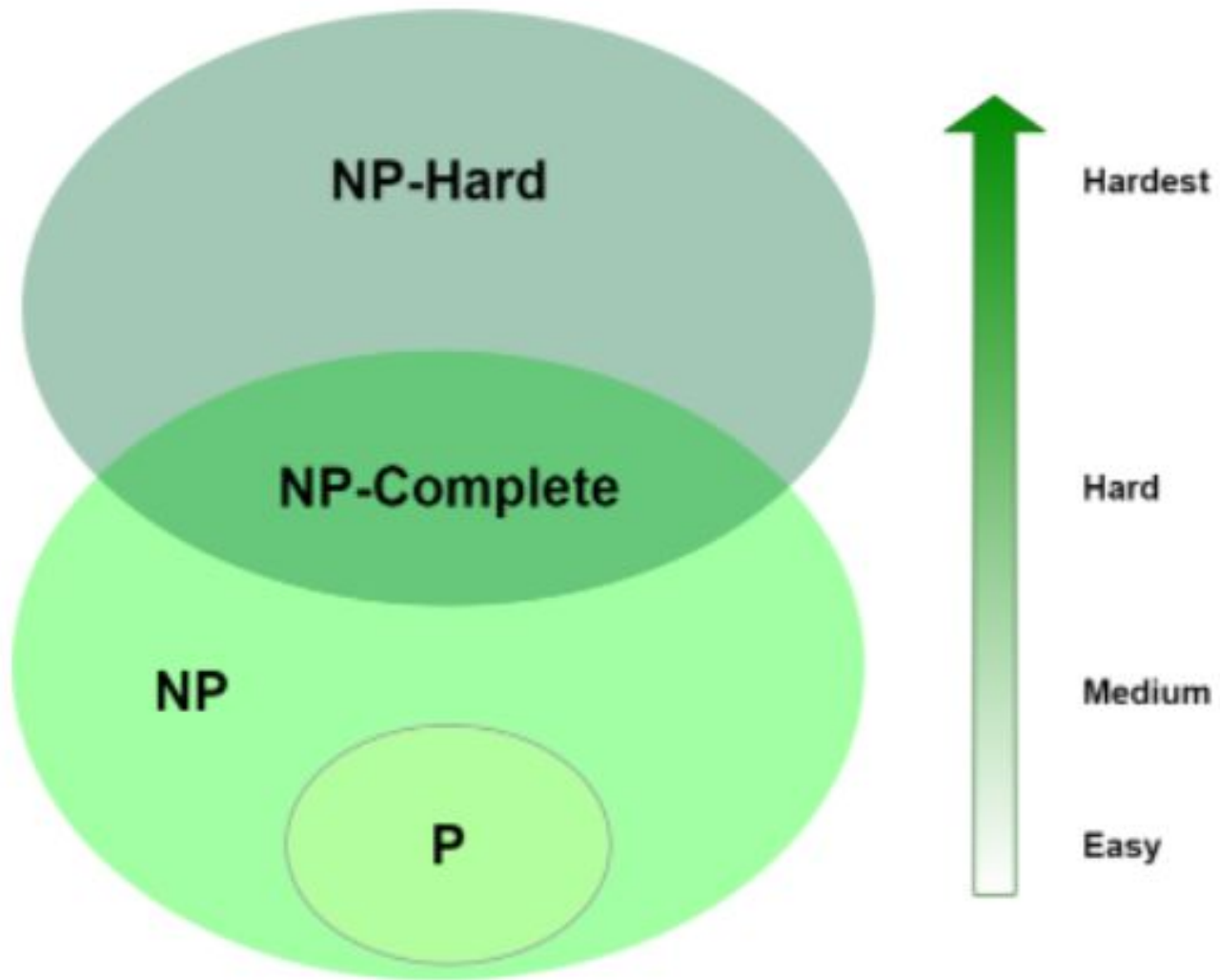
NP Hard

- A problem is NP hard if every problem in NP can be polynomial reduced to it.
- Basically NP hard are optimization problems.



NP Complete

- A problem is called NP complete if it is in NP and it is NP-Hard.
- The intersection of NP and NP hard is NP complete problem.
- Basically NP complete are decision problems.
- Example: If problem A is a Decision problem and problem B is Optimization problem then it is possible $A \in B$.
 - Example : Knapsack decision problem can be converted into knapsack optimization problem.
- All NP-Complete problems are NP-Hard but all NP-Hard problems are not NP-Complete.

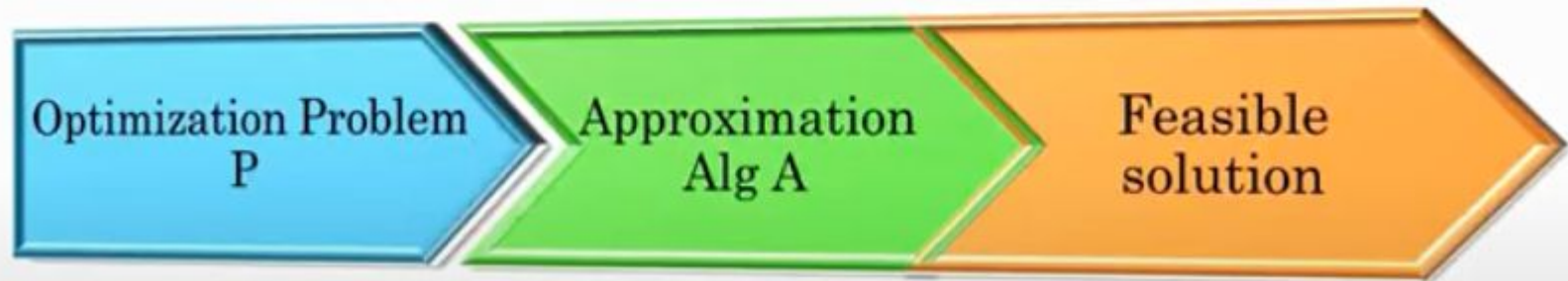


Summary

- Computational complexity
 - Polynomial(P)
- Non deterministic polynomial(NP-Class)
 - NP- Hard
- NP-Complete

Approximation Algorithms

Given an optimization problem P , an algorithm A is said to be an approximation algorithm for P , if for any given instance I , it returns an approximate solution, that is a feasible solution.



Properties of approximation algorithms are

1. Guaranteed to run in polynomial time
2. Guaranteed to get a solution which is close to the optimal solution (near optimal)

Approximation Algorithms

An algorithm that uses random numbers to decide what to do next anywhere in its logic (at least once) is called Randomized Algorithm.

For example, in Randomized Quick Sort, we use random number to pick the next pivot

Approximation Algorithms for NP- Hard

Accuracy ratio: $r(s_a)$

$s_a \rightarrow$ Approximate solution

$f(s_a) \rightarrow$ Value of objective function for solution given by approximation algorithm.

$f(s^*) \rightarrow$ Exact solution of the problem.

Approximation algorithm

- Of course, if we use an algorithm whose output is just an approximation of the actual optimal solution, we would like to know how accurate this approximation is.
- We can quantify the accuracy of an approximate solution to a problem of minimizing some function f by the size of the relative error of this approximation

$$re(s_a) = \frac{f(s_a) - f(s^*)}{f(s^*)},$$

where s^* is an exact solution to the problem. Alternatively, since $re(s_a) = f(s_a)/f(s^*) - 1$, we can simply use the *accuracy ratio*

$$r(s_a) = \frac{f(s_a)}{f(s^*)}$$

as a measure of accuracy of s_a . Note that for the sake of scale uniformity, the accuracy ratio of approximate solutions to maximization problems is usually computed as

$$r(s_a) = \frac{f(s^*)}{f(s_a)}$$

to make this ratio greater than or equal to 1, as it is for minimization problems.

Obviously, the closer $r(s_a)$ is to 1, the better the approximate solution is. For most instances, however, we cannot compute the accuracy ratio, because we typically do not know $f(s^*)$, the true optimal value of the objective function. Therefore, our hope should lie in obtaining a good upper bound on the values of $r(s_a)$. This leads to the following definitions.

Approximation Algorithms for TSP

**Approximation
algorithms**

**Greedy
Approach**

**Minimum-
Spanning-
Tree-Based
Algorithms**

**Nearest-
neighbor
algorithm**

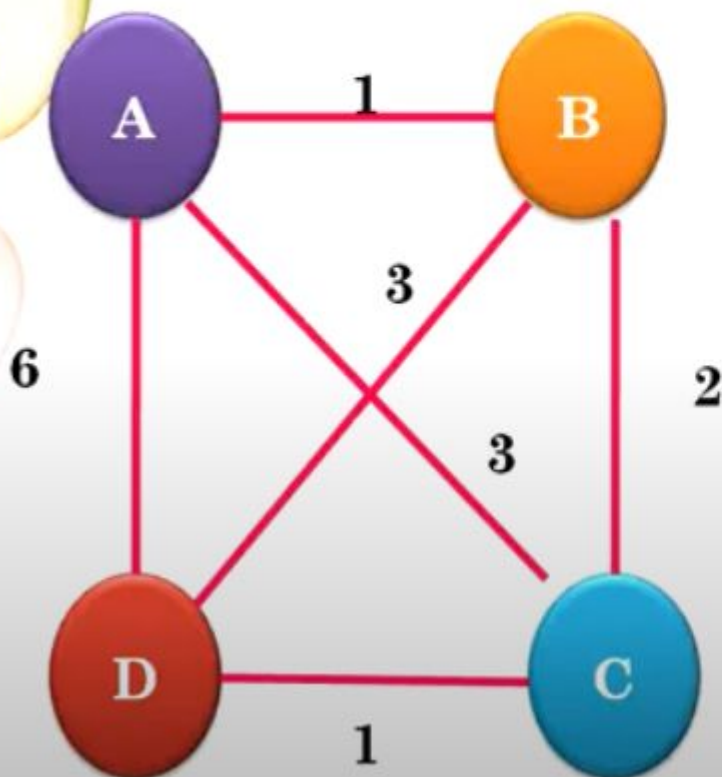
**Multifragment
-heuristic
algorithm**

**Twice-around-
the-tree
algorithm**

**Christofides
Algorithm**

Approximation Algorithms for TSP

Nearest-neighbor algorithm



1. Select a random city.
2. Find the nearest unvisited city and go there.
3. Are there any unvisited cities left? If yes, repeat step 2.
4. Return to the first city.

Nearest-neighbor algorithm for TSP

The following well-known greedy algorithm is based on the *nearest-neighbor*

Heuristic: always go next to the nearest unvisited city.

Step 1 Choose an arbitrary city as the start.

Step 2 Repeat the following operation until all the cities have been visited:

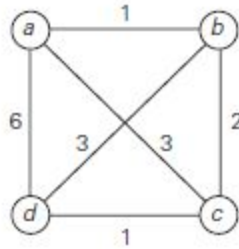
 go to the unvisited city nearest the one visited last (ties can be broken arbitrarily).

Step 3 Return to the starting city.

Nearest-neighbor algorithm for TSP

Example:

Apply nearest-neighbour algorithm for the instance of TSP represented by graph given below. Also find accuracy ratio for the same.

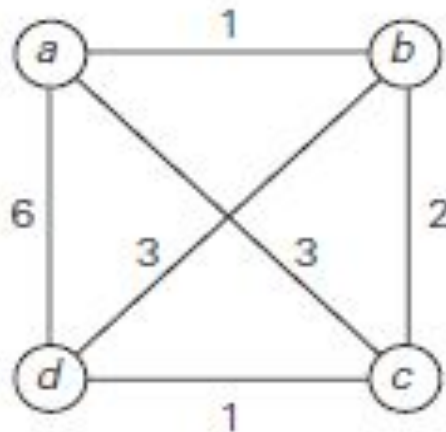


Nearest-neighbor algorithm for TSP

Solution :

1. With a as the starting vertex, apply the nearest-neighbor algorithm
2. The nearest-neighbor algorithm yields the tour (Hamiltonian circuit)

$S_a : a - b - c - d - a$ of length 10.

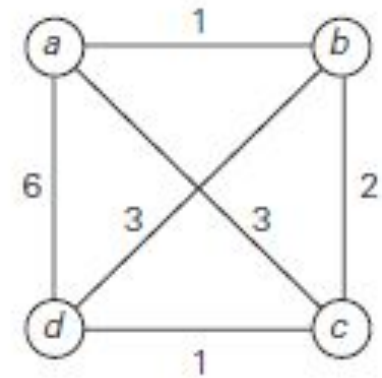


Nearest-neighbor algorithm for TSP

Solution :

3. Now we will apply exhaustive search. That is, we will find all possible tours from node a.

Tour	Length
$a - b - c - d - a$	10
$a - b - d - c - a$	8
$a - c - b - d - a$	14
$a - c - d - b - a$	8
$a - d - b - c - a$	14
$a - d - c - b - a$	10



Nearest-neighbor algorithm for TSP

Solution :

The optimal solution, as can be easily checked by exhaustive search, is the tour

s^* : $a - b - d - c - a$ of length 8.

Thus, the accuracy ratio of this approximation is

$$r(s_a) = \frac{f(s_a)}{f(s^*)} = \frac{10}{8} = 1.25$$

(i.e., tour s_a is 25% longer than the optimal tour s^*).

Twice-around-the-tree algorithm for TSP

Step 1 Construct a minimum spanning tree of the graph corresponding to a given instance of the traveling salesman problem.

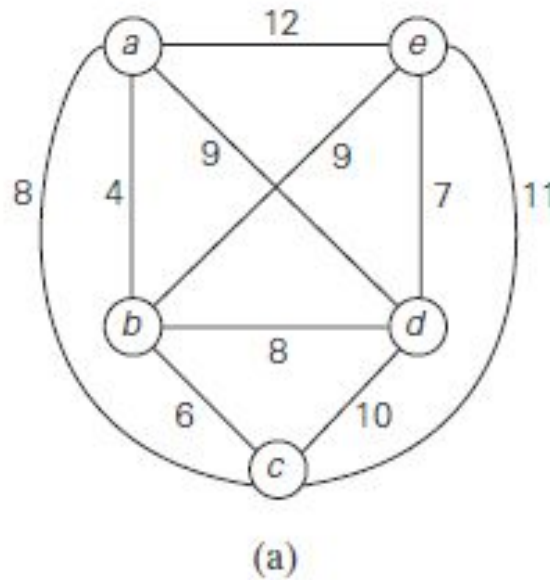
Step 2 Starting at an arbitrary vertex, perform a walk around the minimum spanning tree recording all the vertices passed by. (This can be done by a DFS traversal.)

Step 3 Scan the vertex list obtained in Step 2 and eliminate from it all repeated occurrences of the same vertex except the starting one at the end of the list. (This step is equivalent to making shortcuts in the walk.)

The vertices remaining on the list will form a Hamiltonian circuit, which is the output of the algorithm.

Twice-around-the-tree algorithm for TSP

Example: apply twice around the tree algorithm on the graph given below



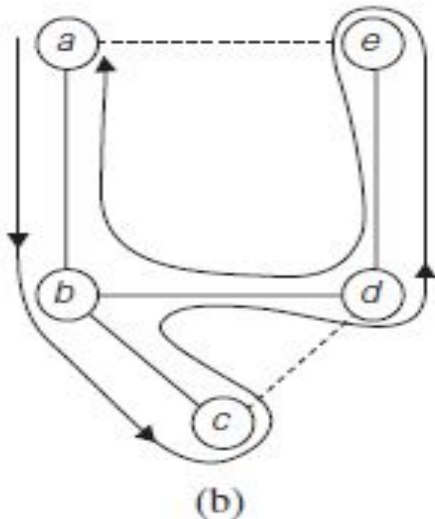
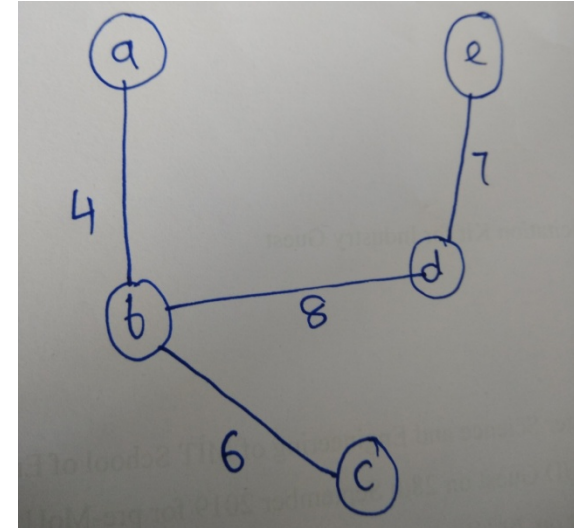
Twice-around-the-tree algorithm for TSP

Solution: A twice-around-the-tree walk that starts and ends at a is $a, b, c, b, d, e, d, b, a$.

Eliminating repetitions *yields the*

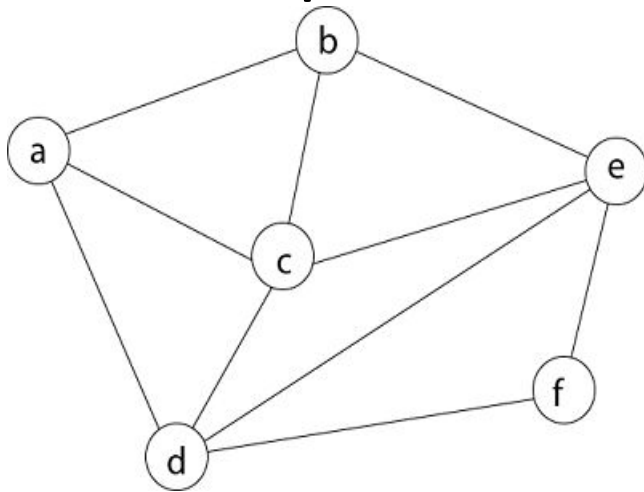
Hamiltonian circuit

a, b, c, d, e, a of length 39.



Hamilton Circuit Problem

- A Hamiltonian cycle is a cycle that contains all vertices in a graph exactly once.
- Example:



Possible Hamilton cycle for
the graph:
a-b-c-e-f-d-a

Hamilton Circuit Problem

- Given a graph $G = (V, E)$ we have to find the Hamiltonian Circuit using Backtracking approach.
- We can assume that if a Hamiltonian circuit exists, it starts at vertex a . Accordingly, we make vertex a the root of the state-space tree .
- The first component of our future solution, if it exists, is a first intermediate vertex of a Hamiltonian circuit to be constructed.
- Using the alphabet order to break the three-way tie among the vertices adjacent to a , we select vertex b . From b , the algorithm proceeds to c , then to d , then to e , and finally to f , which proves to be a dead end.

Hamilton Circuit Problem

- So the algorithm backtracks from f to e , then to d , and then to c , which provides the first alternative for the algorithm to pursue.
- Going from c to e eventually proves useless, and the algorithm has to backtrack from e to c and then to b .
- From there, it goes to the vertices f , e , c , and d , from which it can legitimately return to a , yielding the Hamiltonian circuit a, b, f, e, c, d, a .
- If we wanted to find another Hamiltonian circuit, we could continue this process by backtracking from the leaf of the solution found.

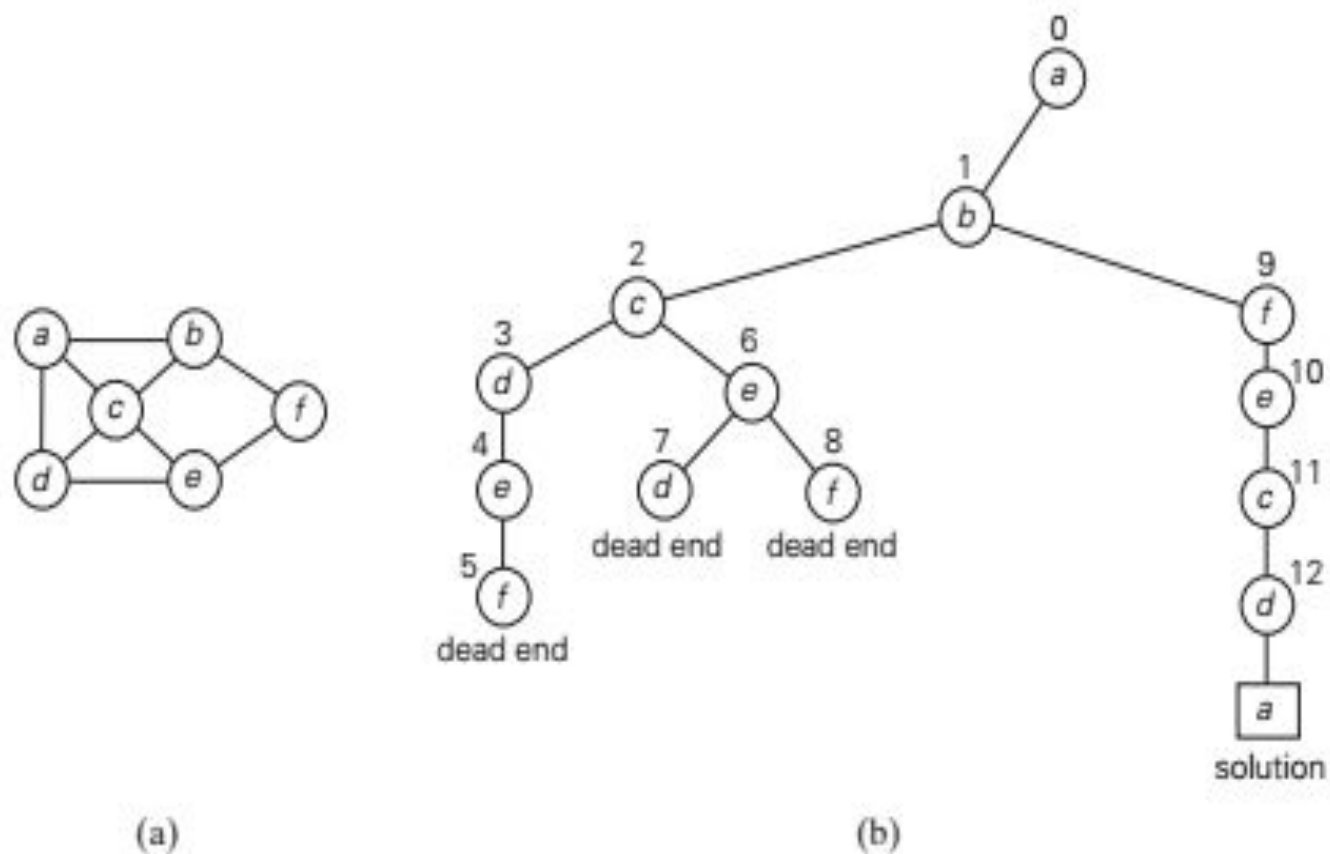


FIGURE 12.3 (a) Graph. (b) State-space tree for finding a Hamiltonian circuit. The numbers above the nodes of the tree indicate the order in which the nodes are generated.

Hamilton Circuit Problem

- If we wanted to find another Hamiltonian circuit, we could continue this process by backtracking from the leaf of the solution found.

Thank You